

An Approach to Generating API Test Scripts Using GPT

Ngoc-Cuong Nguyen
Faculty of Information Technology,
University of Science,
Vietnam National University
Ho Chi Minh City, Vietnam
nncuong19@apcs.fitus.edu.vn

Vu Nguyen
Katalon inc.
Faculty of Information Technology,
University of Science,
Vietnam National University
Ho Chi Minh City, Vietnam
nvu@fit.hcmus.edu.vn

Quang-Huy Bui
Faculty of Information Technology,
University of Science,
Vietnam National University
Ho Chi Minh City, Vietnam
bqhuy19@apcs.fitus.edu.vn

Tien N. Nguyen
Katalon inc.
University of Texas at Dallas
Dallas, Texas, USA
tien.n.nguyen@utdallas.edu

ABSTRACT

As more software systems publish and use web services or APIs today, automated API testing is an important activity to help effectively ensure the quality of software services before they are released for their usage. Generating test scripts and data is a crucial step to perform API test automation successfully. In this paper, we propose an approach leveraging GPT, a large language model, and API's Swagger specification to automatically generate test scripts and test data for API testing. Our approach also applies GPT's self-refining with the feedback by executing tests on Katalon. We evaluate our proposed approach using a data set of seven APIs consisting of 157 endpoints and 179 operations. The result shows that while our approach generates fewer test scripts and data inputs, it can cover more successful status codes of 2xx than a state-of-the-art tool. This result suggests that leveraging the ability of GPT as a large language model to interpret API's Swagger specification has the potential for improving the efficacy of generating test scripts and data for API testing.

CCS CONCEPTS

• Software and its engineering → Software maintenance tools.

KEYWORDS

Restful APIs, Automated API Testing, Large Language Models (LLMs), GPT, Prompt Engineering, API Test Data, API Test Script, Katalon

ACM Reference Format:

Ngoc-Cuong Nguyen, Quang-Huy Bui, Vu Nguyen, and Tien N. Nguyen. 2023. An Approach to Generating API Test Scripts Using GPT. In *The 12th*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SOICT 2023, December 07–08, 2023, Ho Chi Minh, Vietnam

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0891-6/23/12...\$15.00
<https://doi.org/10.1145/3628797.3628947>

International Symposium on Information and Communication Technology (SOICT 2023), December 07–08, 2023, Ho Chi Minh, Vietnam. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3628797.3628947>

1 INTRODUCTION

Automated Restful API testing plays a important role in software development today, offering a streamlined approach to comprehensively verify the behavior of web services or APIs of software systems. However, challenges arise in ensuring the flexibility and semantic coherence of generated test data, leading to complexities in managing various constraints, dependencies, and semantic meanings associated with each parameter [16]. Central to automated API testing is the generation of test scripts and data to validate and expose errors in APIs. Many approaches have been introduced to generate test scripts and data for automated API testing [1–4, 6–8, 11, 12, 14, 16, 21]

As the Swagger or OpenAPI specification¹ has become a standard method for documenting web APIs, many approaches use it as a basis and key information for automatically generating test scripts and data for API testing [1, 2, 6, 7, 9, 14, 16]. Using natural language descriptions of the Swagger specification, several approaches infer constraints and rules for API operation call sequences and input parameters [10, 14, 17].

This paper proposes an approach using GPT-3.5 turbo [15] (hereafter GPT for short) to automatically generate test scripts and data for API testing from the Swagger specification. Our approach analyzes a Swagger specification file as an input, applies GPT's prompt engineering to generate test scripts and data [20], executes test scripts, and performs a self-refinement or closed-loop method [13] with the feedback from Katalon's Run-time Engine². Generated test scripts and data are executed and assessed against API operations and response codes defined in the Swagger specification. To the best of our knowledge, this is the first study leveraging GPT, prompt

* Corresponding: Vu Nguyen (nvu@fit.hcmus.edu.vn)

¹<https://swagger.io/resources/open-api/>

²<https://katalon.com>

engineering, and self-refinement for automated API's test scripts and data generation using Swagger specification.

We evaluate the proposed approach and comparatively assess its performance against the state-of-the-art tool RestTestGen [16] using a data set of seven open-source APIs consisting of 157 endpoints and 179 operations. We use the percentage of HTTP status codes (such as 200, 400, and 500) covered and execution time as two primary measures of the effectiveness of the API test generation model. The results obtained show that the proposed approach generates test scripts and data to cover more successful response codes (e.g., 2xx) than does RestTestGen while it **covers fewer failure status codes** (4xx and 5xx). In terms of execution time, both approaches are comparable. Another noticeable result is that our approach uses GPT to guide the process of test generation, needing fewer test cases to cover more status codes while RestTestGen generates a number of mutations which require many test cases to cover status codes.

The remainder of this paper is organized as follows. Section 2 provides an overview of related work in the field of automated API testing. Section 3 details the methodology and experimental setup used in our study. Section 4 presents our findings and result analysis. Finally, Section 5 concludes the paper by summarizing the contributions and discussing potential avenues for future research.

2 RELATED WORK

In today's fast-paced software development landscape, the evolution of automated API testing tools such as RESTest, ResTestGen, EvoMaster, Toga, RestCT, Resler, and Morest [3–5, 12, 14, 16, 18, 21] signifies a profound transformation in the way we ensure software quality. These tools empower developers and testers with innovative solutions to comprehensively assess an API's functionality, uncover potential issues, and streamline the testing process.

RESTest's reliance on OpenAPI Specification (OAS) greatly simplifies the generation of test cases, providing a holistic view of an API's behavior [14]. ResTestGen's intelligent inference of relationships between endpoints facilitates the creation of purposeful test sequences, encompassing both success and error scenarios [16]. EvoMaster's integration of evolutionary algorithms not only bolsters adaptability but also reinforces the reliability of generated test cases, catering to both white-box and black-box approaches [3, 21].

Toga employs advanced neural technology to address persistent challenges related to missing documentation and implementations, ultimately boosting the accuracy of test oracles [5]. Meanwhile, RestCT's systematic testing methodology brings about a significant enhancement in test coverage and efficiency, often leading to the discovery of previously overlooked bugs [18].

Restler offers a lightweight and straightforward approach, ideal for small to medium-sized projects, simplifying REST API testing with fuzzing mechanism[4]. Morest, on the other hand, provides a user-friendly interface for quick and efficient API tests, making it accessible to both seasoned testers and developers [12].

These tools collectively exemplify the ongoing innovation in automated API testing, highlighting its pivotal role in modern software development practices. As the software industry continues to advance, these tools stand as crucial allies in the quest for robust

and dependable APIs, ensuring that software applications meet the requirements for quality and reliability.

3 METHODOLOGY

Our research combines techniques from related works with the capabilities of the GPT model to present a responsible implementation of prompt engineering for test data generation in Automated API testing. By investigating API specification analysis, prompting design, strategies, and integration with **Katalon**, a powerful automated testing tool, we present a comprehensive and structured approach for generating test data and executable test scripts in **Katalon**. Our method takes the form of an automated system comprising several steps

3.1 Testing Order Specification

To ensure a comprehensive evaluation of the under-test endpoints, we design a testing sequence that considers the order of testing different HTTP methods. By following this sequence, we can effectively assess the functionality and performance of the endpoints while accounting for dependencies and preconditions among operations. To enhance the specificity of the methodology, the data obtained from previous test executions can be utilized as preexisting values for subsequent method testing processes. This approach enables the optimization of test performance by reducing redundant data generation or retrieval. The specific testing order in our approach is as follows:

- **GET all:** Retrieve all data to understand content and structure of the endpoint.
- **POST:** Create new data to test insertion and data accuracy.
- **GET by URL parameter:** Retrieve specific data subset using defined parameters to assess filtering accuracy.
- **PUT by URL parameter:** Update existing data via specific URL parameters to evaluate modification accuracy.
- **DELETE by URL parameter:** Remove targeted data using URL parameters to test data deletion and accuracy.

3.2 API Specification Analysis

Our testing approach strictly follows the black-box testing technique, which focuses solely on the external behavior and functionality of the system. By adopting this methodology, we ensure that our evaluation is independent of the internal implementation details of the API. As such, we rely exclusively on the API specification provided in OpenAPI swagger format as the input for our testing process.

The detailed process of the analysis process is described in Figure 1. In the initial phase, the Swagger specification is subjected to a thorough analysis, enabling us to extract and comprehend the necessary information required for the subsequent automated testing procedures. This analysis involves parsing the swagger file to identify endpoints, their associated HTTP methods, input parameters, and expected responses.

During the testing process of scenarios that assert successful outcomes for the requests, it is possible that certain related endpoints need to be executed as preconditions for successful testing. These related endpoints provide the necessary data or context that the testing endpoint requires to achieve the expected status code.

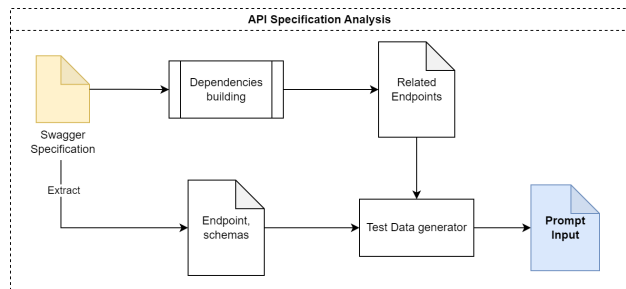


Figure 1: API Specification Analysis

An essential aspect of this testing process involves analyzing the OpenAPI swagger specification to identify the cross-API dependencies or related endpoints associated with the under-test endpoint [16].

The identified cross-API dependencies, along with the extracted information from the endpoint and related endpoints, collectively serve as the input information for generating the corresponding prompt for each test case.

By leveraging the combined input information, our approach facilitates the generation of tailored prompts that consider the inter-dependencies among endpoints, ensuring thorough testing coverage and accurate evaluation of the endpoint's performance and compliance with the API specification.

3.3 Prompt Generation

3.3.1 Prompting strategies and components. Utilizing the input obtained from the pre-processing stages, we proceed with the generation of prompts to facilitate the generation of API test data for various endpoints and corresponding status codes, employing the principles of the GPT model. To ensure optimal test data generation for each specific scenario, we implement the "divide and conquer" methodology, wherein we create multiple versions of instructions, techniques, and prompt designs tailored to different specific scenarios combined with a variety of techniques and strategies recommended by OpenAI³ in prompt engineering implementation.

The strategies for different scenarios are inferred through rigorous observation of the API testing process and the underlying criteria. To ensure the completeness of our prompts, we create diverse versions for each scenario. Our innovative system then intelligently combines these versions, forming comprehensive prompts that optimize the performance of our GPT model. By employing this method, our prompts become highly specific, enabling the generation of coherent, context-aware, and precise responses.

To enhance the outcome of the prompts, we have explored and implemented a range of techniques and strategies endorsed by GPT. Our aim is to bolster the GPT model's comprehension of natural language, thereby refining the prompt generation process and elevating the model's capacity to produce coherent, context-aware, and precise responses [19]. Following these strategies, the design of our prompts in several scenarios incorporates the following components:

³<https://openai.com/>

Test case scenario: This component provides instructions for the GPT model regarding specific testing scenarios. It includes details about the endpoint under test, expected status codes, testing strategies, and related endpoints if applicable. This helps the GPT model understand the task and generate suitable responses.

Data generation rules: Test data generated must adhere to defined rules, which may vary based on endpoint dependencies. Rules distinguish scenarios with and without cross-endpoint dependencies.

Extracted information from swagger analysis: Relevant information from Swagger analysis is organized in the prompt. This includes details for testing endpoints and schemas of the parameters and request body. This organization ensures clarity, aiding GPT in producing accurate and relevant responses aligned with testing objectives.

Expected result template: Examples of expected responses from relevant endpoints are provided. Nominal and error test case templates specify these responses. Nominal cases indicate requests with expected outcomes (status codes in the 2xx range), while error cases signify unexpected results. Expected responses are enclosed within `<code></code>` tags for extraction. This rule is also outlined in the general prompt guidelines.

Execution feedback if existed: After test case execution, a self-feedback mechanism is integrated into the GPT model's prompting process. This optimizes test coverage based on categorized feedback scenarios, such as non-matching status code assertions. Feedback is appended from runtime's execution log to refine response generation.

3.3.2 Predefined Prompt Scenarios. With the objective of enhancing the coverage of status codes in the testing process, our approach defines common scenarios based on the definition of status code groups from OpenAPI.

By utilizing the predefined common scenarios, our methodology involves subjecting each API endpoint to comprehensive testing. This entails iteratively evaluating the endpoint's behavior across all scenarios, using the corresponding prompts.

To provide further clarity, each endpoint under test will be subjected to testing through all the predefined scenarios indicated in Figure 2. Each scenario is associated with its own prompt and will be presented to the GPT model using multi-shot prompting with execution feedback. The data response of each scenario is then collected to achieve complete coverage of all the relevant status codes in the testing of the endpoint. This meticulous approach ensures a comprehensive evaluation of the API's behavior and robustness under various conditions.

As a comprehensive explanation of Figure 2, we present the list of distinct scenarios meticulously defined by our approach, each accompanied by its corresponding predefined strategies:

Request with Successful Response: Generating requests with successful outcomes, achieving 2xx status codes, and precise adherence to the test endpoint schema.

Request with Not Found Error: Creating requests that result in "Not Found entity" error responses (typically 4xx status codes), involving non-existent query parameters.

Unprocessable Request: Generating requests that are inherently unprocessable due to encoding/decoding system issues.

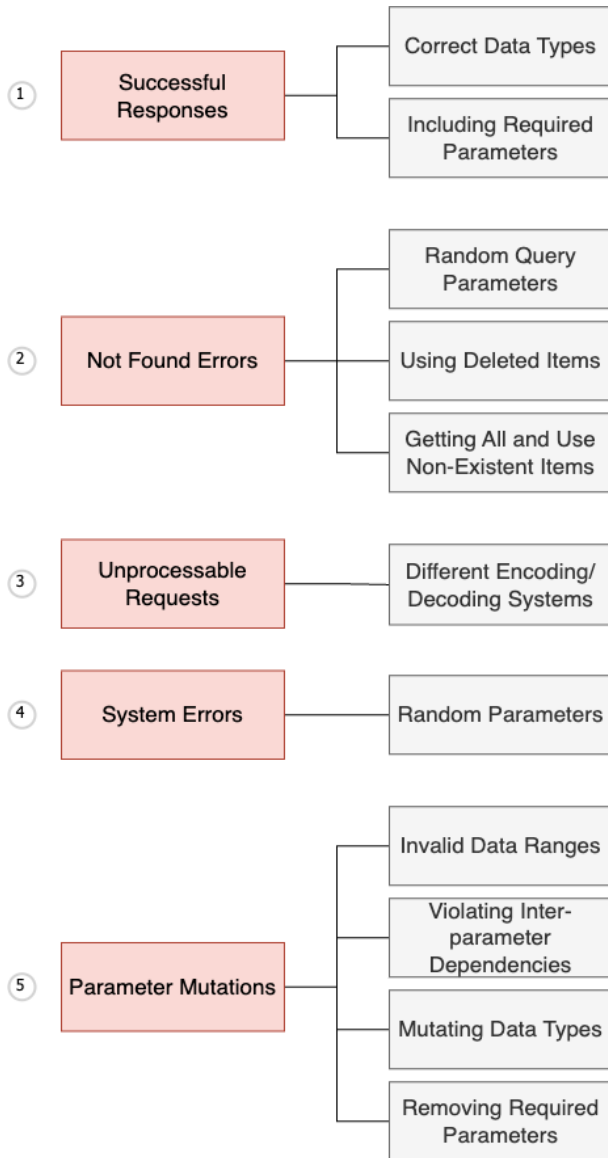


Figure 2: Scenarios-Oriented Prompts

Request with Parameters Mutation: Intentionally producing invalid requests with semantic errors or validation issues related to parameters, causing 4xx status codes. Utilizing the ResTestGen method for request body mutation.

Request with System Error: Generating requests leading to server-side system-level errors, typically resulting in 5xx status codes (e.g., 500). Triggering exceptional conditions or errors in the server's backend.

In various instances and within specific contexts, the swagger API specification may include definitions of specific status codes that do not align with the predefined common scenarios employed in our approach. To enhance the optimization of status code coverage, we have implemented a greedy method to encompass these status codes. For each operation, upon completion of iterations through

all the predefined scenarios, our system undertakes a search for the status codes that remain uncovered within the recorded status codes resulting from the requests made in our process.

3.4 Test Data Generation

3.4.1 Prompting Execution. In our approach, we construct a comprehensive message by combining a system message that sets the persona for the GPT model and a corresponding prompt tailored to the specific testing scenario of the endpoint and status code.

The system message serves as an introduction and context for the GPT model, providing it with a persona to adopt during the conversation. This persona helps guide the model's responses and align them with the desired style and tone of the testing process. The construction of this persona is intricately linked to the precise tasks we intend the GPT model to perform in particular instances. In the context of our application, focused on the generation of test data for automated API testing.

Alongside the system message, we design a prompt specific to the testing endpoint and status code being evaluated. This prompt includes relevant information about the scenario, such as the API endpoint, the expected status code, and any additional instructions for generating suitable test data or verifying the response.

By combining the system message with the scenario-specific prompt, we create a cohesive message that ensures the GPT model understands the context, requirements, and constraints of the testing scenario. This approach enhances the model's ability to generate relevant and accurate responses, aligning with the testing objectives and facilitating effective communication between the system and the model.

We utilize the message to invoke the API in order to initiate the chat completion process using the specified configuration for the GPT-3.5-turbo-16k-0613 model, as outlined in Figure 3. This configuration includes the specified model, messages containing the prompts, and the temperature parameter. Subsequently, a formal request is made to OpenAI's API:

```

response = openai.ChatCompletion.create(
    model='gpt-3.5-turbo-16k-0613',
    messages=messages,
    temperature=0.5,
)
return response.choices[0].message
  
```

Figure 3: Python code for using the GPT-3.5 Turbo model.

Considering that the desired response from the GPT model entails a script component along with an explanation generated by GPT, particularly when retrieving response data from previous test cases and when GPT is expected to learn from its own answers, we will extract the test data for API testing by specifying the `<code>` tags to restrict the GPT model's placement of the test data. This approach allows us to segregate the test data from the generated content, facilitating its extraction and utilization.

The extracted test data will then be incorporated into the integration process with **Katalon**, enabling us to verify the functionality

and accuracy of the API. Additionally, the extracted data will serve as a valuable resource for executing other test cases, contributing to a comprehensive assessment of the GPT model's performance and capabilities.

3.4.2 Request and Response Storage. To optimize resource allocation and streamline our testing process, we have devised a systematic approach that involves two distinct types of files:

Successful Nominal Requests for Future Mutation: This process involves capturing and storing the details of successful test case requests for specific endpoints. By preserving the original request structure, including headers, parameters, and payload, it eliminates the need to recreate them in subsequent test cases. These stored requests serve as a foundation for generating mutated test data, streamlining the exploration of various scenarios.

Successful Nominal Responses for Specific Conditions and Other Endpoints: In a complementary effort, successful test case responses are curated and stored separately. These stored responses are valuable resources that can be reused in test cases for other endpoints requiring the response data from the current testing endpoint as input. This approach saves time and effort while ensuring consistency in the testing process.

By combining the stored successful nominal requests and responses, we establish a framework that promotes resource optimization and testing efficiency. The stored requests facilitate the generation of mutated test data, enabling thorough exploration of various scenarios. Simultaneously, the stored responses contribute to the reusability of preexisting values, ensuring consistent and reliable testing outcomes across different endpoints and conditions.

3.5 Execution Feedback

In order to enhance our approach for generating successful test cases, we employ a multi-shot prompting technique, which leverages both previous log data and the model's responses as execution feedback. This approach significantly improves the efficiency and effectiveness of our testing process.

Execution feedback plays a crucial role in addressing compile errors and runtime errors encountered during the testing phase. When GPT generates a test case, it analyzes the log generated during previous execution. This log contains information about any errors or exceptions that may have occurred during the test run.

By processing the execution log, the model can identify and categorize errors, such as syntax errors, variable mismatches, or unexpected runtime behavior. This analysis allows the model to gain insights into the root causes of these errors and generate a new output that specifically addresses the detected issues.

For instance, if the model encounters a compile error due to a mismatched data type, it can use the information from the log to refine the input or make necessary adjustments to the code to rectify the issue. Similarly, in the case of a runtime error, the model can analyze the log to pinpoint the source of the error and adapt the test case accordingly, ensuring that it exercises the problematic code paths.

3.6 Katalon Integration

The information extracted from the GPT model's response serves as essential API test data, which will subsequently be utilized for

integration with automated test frameworks. This integration is designed to facilitate calls to the API server while concurrently verifying the response in accordance with the status code assertion. Furthermore, the execution log generated by the framework will be harnessed to facilitate the self-feedback mechanism integral to GPT's prompt engineering.

In our research endeavor, we have chosen to employ **Katalon**, a robust automated testing tool renowned for its effectiveness. By harnessing the amalgamation of **Katalon**'s diverse features in conjunction with our proposed methodology, the expected outcomes of this process encompass the generation of an execution log for the self-feedback mechanism and the results of the test script execution. The precise configuration of the **Katalon** integration step within our system is meticulously elucidated in the accompanying Figure 4.

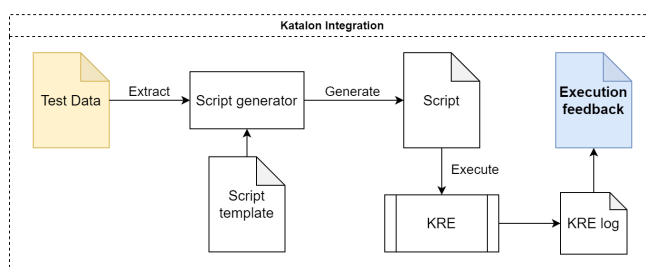


Figure 4: **Katalon** Integration

3.6.1 Object Repository. The TestObject functionality within Katalon Studio enables the definition and organization of objects or elements, such as web elements, mobile objects, or API objects, that are interacted with during test automation. These objects encompass a range of elements including buttons, text fields, dropdowns, links, and more.

By utilizing the TestObject feature, the attributes and properties of these objects can be stored within a centralized repository, thereby simplifying the maintenance and updating of test cases. This approach contributes to the reduction of redundancy and the enhancement of test script reusability. For instance, consider the PetStore service's operation `GET /user/login`, which necessitates two parameters, namely `username` and `password`, within the query path. In the realm of **Katalon** Studio, this operation is encoded as a TestObject labeled as `loginUser`, as depicted in Figure 5.

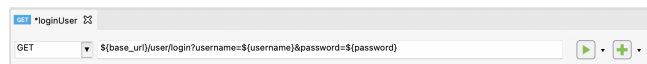


Figure 5: Sample TestObject in **Katalon** Studio

The aggregation of TestObjects is denoted as the Test Repository, a construct that delineates the architecture of an Application Under Test (AUT). The transformation of a Swagger Specification into a Test Repository is an automated process facilitated by **Katalon**

Studio; however, users are required to manually import the Swagger Specification via the **Katalon** UI, subsequently configuring global variables.

3.6.2 Katalon Runtime Engine. The **Katalon** Runtime Engine (KRE) constitutes an integral facet of **Katalon** Studio. This engine allows executing automated tests and test suites within a command-line without the need of launching **Katalon** Studio. Our approach uses KRE to execute generated test suites and obtain execution logs for further analysis.

3.6.3 Katalon Test Script Template. Upon extracting the API test data from the GPT response, the data undergoes amalgamation with a predefined Groovy test script combined with the test data from the GPT model as outlined in Figure 4 that is executable within the **Katalon** framework. Diverse test case scenarios are accommodated by separate test scripts, each equipped with specific assertions tailored to their respective scenarios. The specific assertions for these varying scenarios are outlined as follows:

- Request with the successful responses:

```
assert 200 <= response.getStatusCode();
assert response.getStatusCode() < 300;
```

- Request with the client errors:

```
assert 400 <= response.getStatusCode()
assert response.getStatusCode() < 500
```

- Request with the server errors:

```
assert response.getStatusCode() >= 500
```

The provided example represents a standardized test script template tailored for a specific 4xx status code test case. Within this template, the assertion is adjusted in accordance with the particular scenario, while all other syntax elements are predefined to ensure the test script's executability within the **Katalon** framework, thereby mitigating potential syntax errors. Each individual test case incorporates distinct test data and pertinent information, culminating in a fully executable test script when populated with the requisite specifics. In the test script, the logic of storing successful requests and responses is also implemented.

By employing this structured approach, we can efficiently create test scripts for different test case scenarios, effectively validating the desired status code responses and their respective outcomes with precision and reliability. Through the operations with KRE, the test script results are recorded in the execution log, providing comprehensive execution feedback for our process, as illustrated in the concluding section of Figure 4.

4 EXPERIMENT

4.1 Dataset

We evaluated our approaches based on seven open-source API services as shown in Table 4.1. Our target RESTful services are diverse in terms of complexity, features, and number of operations.

Service	Endpoints	Operations
Petstore	13	19
Proshop	7	16
API Guru	7	7
CanadaHolidays	7	7
BillsApi	11	11
RealWorld	12	19
Jikan	100	100

Table 1: Services under test

These services are collected from multiple sources whose server are functional. We use their existing Swagger specifications without modification.

4.2 Evaluation Baseline

We used the state-of-the-art tool RestTestGen as a baseline to evaluate our model. RestTestGen applies the black-box technique which generates Operation Dependency Graph (ODG) to construct dependencies between operations. Using ODG, RestTestGen produces sequence operation calls to reflect the dependencies between operations and their HTTP methods [16]

4.3 Evaluation Criteria

We evaluate our approach based on two criteria which will be comprehensively presented in the following sections of this section:

Status code coverage: The assessment of response status code coverage exemplifies the testing tool's capability to generate appropriate test data and test cases in adherence to the Swagger specification interface. The outcomes of our research analysis primarily concentrate on the comprehensive coverage of each category of status codes as outlined in the OpenAPI documentation. Moreover, we have diligently collected and analyzed the detailed results of status codes that are not explicitly documented in the API specification. This comprehensive approach allows us to gain a holistic understanding of the API's behavior and potential edge cases beyond the officially documented status codes. Consequently, our approach covers a broader range of cases than those documented in the Swagger specification.

Success Test Cases Rate: This criterion involves collecting data on the proportion of successful test cases in nominal scenarios. The purpose of this evaluation is to compare the efficiency of the approach being studied against an alternative approach in generating test data that align with the Swagger specification. A higher success rate indicates that the approach is better at producing test cases that meet the specified criteria.

Execution Time: The execution time criterion focuses on measuring the time it takes for the testing approach to generate and

execute the test cases. A shorter execution time is generally preferred, as it indicates quicker testing and validation of the API's behavior.

Cost: This metric is used to evaluate the cost for a complete test process for a service under test.

5 RESULT

5.1 Status Code Coverage

Service	Op.	Operations Covered		Coverage Percentage (%)	
		RTG	Ours	RTG	Ours
Petstore	15	11	15	73	100
Proshop	16	14	16	88	100
API Guru	7	5	5	71	71
CanadaHolidays	6	6	6	100	100
BillsAPI	11	8	9	71	82
RealWorld	19	16	16	84	84
Jikan	100	85	92	85	92

Table 2: 2xx Status Code Coverage

Table 2 indicates the number of successful status codes (2xx) which are documented in the swagger specification and covered during the testing process by *RestTestGen* and our approach. Overall, our approach's result is better compared to *RestTestGen*'s, especially in *PetStore* and *Proshop* service, our method could produce successful requests to all operations which are documented. In detail, our approach outperforms *RestTestGen* in its coverage of 2xx status codes due to the following reasons:

- We have identified a potential bug in *RestTestGen* concerning the serialization of JSON data payload in POST requests, lots of test cases resulted in 422 - Unprocessable Entity error code since the redundant comma in POST body
- The GPT model exhibits a remarkable ability to grasp the semantic meaning of parameters. As a result, during the nominal test cases, GPT is proficient in generating API test data with semantic correctness, surpassing the dependency on data types alone, which is the case with *RestTestGen*. This is especially evident when dealing with scenarios involving numerous parameters and high complexity, where the GPT model achieves high accuracy. For instance, in our approach covering the *PetStore* API service, we achieved a 100% coverage of status codes in the 2xx range, whereas *RestTestGen* only achieved 73% coverage.
- In our approach, every endpoint being tested is associated with multiple dependencies from related endpoints, which are identified during the cross-endpoint dependencies analysis phase. These dependencies may originate from one or multiple related endpoints. While *RestTestGen* is proficient in handling dependencies from a single endpoint, the GPT model demonstrates a superior capability in managing dependencies from multiple endpoints that are connected to the testing endpoint. This indicates the GPT model's heightened awareness and understanding of dependencies, which surpasses that of *RestTestGen*

Service	Op.	Operations Covered		Coverage Percentage (%)	
		RTG	Ours	RTG	Ours
Petstore	20	16	18	80	90
Proshop	15	15	15	100	100
API Guru	0	0	0	N/a	N/a
CanadaHolidays	2	2	2	100	100
BillsAPI	19	14	12	74	63
RealWorld	16	2	4	13	25
Jikan	100	47	24	47	24

Table 3: 4xx Status Code Coverage

The ability to generate client-side errors is relatively comparable between *RestTestGen* and our approach, as depicted in Table 3. Despite the similar overall coverage of 4xx status codes, we conducted a thorough analysis of the report to discern the relative strengths and weaknesses of both our approach and *RestTestGen*

Our disadvantages:

- *RestTestGen* exhibited a substantial volume of test cases for each operator, particularly notable in the case of the *updatePet* operator within the *PetStore* service. Specifically, *RestTestGen* generated over 213 test cases for this operator, while our approach generated only 58 test cases. This discrepancy indicates that *RestTestGen* employs extensive strategies for generating error test cases, leading to a more comprehensive coverage of potential scenarios.

Our advantages:

- *RestTestGen* refrains from applying the Error Generator to operations that did not receive at least one successful response. In contrast, our approach demonstrates greater flexibility as we have implemented a strategy that generates various scenarios to achieve the desired target status codes. This adaptability allows our approach to explore a wider range of test cases, even for operations that might not have resulted in successful responses during the initial testing phase.

- Large language models, particularly the GPT model, demonstrate the capacity to comprehend the implications of swagger specifications. Leveraging the descriptive information provided in the response status code, the GPT model proficiently generates customized test data that aligns with the logic of RESTful services. A noteworthy example can be observed in the *PetStore* service, where the status code 400 is utilized in different operations, but with distinct meanings based on the underlying business logic. Remarkably, the GPT model can grasp the significance of these response details and generate suitable test data to achieve the desired target status code for each specific scenario. This ability showcases the GPT model's effectiveness in producing test data that aligns with the intricacies of the service's behavior and logic.

5.2 Success Test Cases Rate

The success test cases rate serves as a crucial measure of the testing approach's ability to generate functional and valid test cases that adhere to the API's specifications. A higher success rate indicates that the approach is more proficient in producing test data that aligns with the expected behavior of the API. Table 4 presents a comparison of the success test cases rates (Rate) between the *RestTestGen* approach and our proposed approach. The success test

Application	RestTestGen			Our Approach		
	N_{tc}	N_{stc}	Rate	N_{tc}	N_{stc}	Rate
PetStore	738	86	0.12	28	15	0.54
ProShop	700	98	0.14	21	16	0.76
APIGuru	390	117	0.30	8	5	0.63
CanadaHolidays	86	77	0.90	8	6	0.75
BillsAPI	1147	114	0.10	19	9	0.47
RealWorld	481	114	0.24	45	16	0.36
Jikan	2296	771	0.34	372	92	0.25

Table 4: Successful Frequency

Application	RestTestGen		Our Approach	
	N_{tc}	Execution Time (Hour)	N_{tc}	Execution Time (Hour)
PetStore	1977	00:30	207	00:56
ProShop	2314	01:05	138	00:40
APIGuru	419	00:06	40	00:17
CanadaHolidays	795	00:05	49	00:23
BillsAPI	2776	00:28	68	00:31
RealWorld	2210	00:21	207	01:25
Jikan	12458	05:24	665	03:20

Table 5: Execution Time

cases rate is calculated by the number of total test cases in nominal N_{tc} divided by number of successful test cases N_{stc}

In the context of the success test cases rate, the following observations can be made:

RestTestGen Results: The success rates achieved by RestTestGen vary across different applications. Notably, the success rates for most of the applications are relatively low, ranging from 0.10 to 0.34. This indicates that RestTestGen struggled to generate successful test cases that align with the API specifications for these applications. Notably, the PetStore, ProShop, and BillsAPI applications exhibited particularly low success rates, highlighting the limitations of the RestTestGen approach in accurately representing the intended API behaviors.

Our Approach Results: The success rates achieved by our approach present a more promising outlook. Across the different applications, the success rates range from 0.25 to 0.76. Notably, our approach outperforms RestTestGen in several cases. For instance, in the PetStore, ProShop, APIGuru, and CanadaHolidays applications, our approach consistently achieves higher success rates. These results indicate that our approach demonstrates an improved ability to generate test cases that result in successful responses based on the API specifications.

5.3 Execution Time

The execution time of the test case generation and execution process is an essential factor to consider in evaluating the efficiency and effectiveness of a testing approach. In our comparative analysis, we measured the execution time for both RestTestGen and our approach for the tested services. The results are summarized in Table 5.

Table 5 provides a comparison of the execution time between RestTestGen and our approach for each application. Overall, our approach has a longer execution time compared to RestTestGen for most applications. However, it is important to note that our approach achieves comparable or better coverage of status codes, as discussed in the previous section.

Our approach utilizes a large language model, specifically the GPT model, to generate test cases. The generation process involves multiple iterations and can be computationally expensive, resulting in longer execution times.

Despite the longer execution time, our approach offers several advantages over RestTestGen, such as improved coverage of status codes and the ability to generate test cases that align with the behavior and logic of the API. These advantages justify the additional time required for execution and demonstrate the effectiveness of our approach in generating comprehensive and accurate test cases

6 CONCLUSION

This paper has presented an approach to generating test scripts and data for API service testing using GPT. The approach takes Swagger specification as the main input, uses the **GPT-3.5 turbo** model through prompt engineering and self-refining for generating scripts and data. Our evaluation using the data set consisting of seven RESTful APIs shows that the proposed approach generates test scripts and data to cover more successful status codes (2xx) than does the state-of-the-art RestTestGen. But it **covers fewer failure status codes** (4xx).

Our approach differs from RestTestGen in generating test scripts and data. RestTestGen performs various mutations while our approach uses GPT with prompts to generate test scripts and data intended to cover endpoints, operations, and status codes defined in the Swagger specification. As a result, our approach achieves the objective of covering endpoints, operations, and status codes with fewer tests. On the other hand, our does not cover failure status codes (4xx and 5xx) as well as RestTestGen as these failure status codes may not be defined sufficiently in the specification.

Our approach has several limitations. The efficacy of our approach is contingent upon the lucidity, specificity, and thoroughness of the descriptions attributed to endpoints, status codes, and corresponding schema in the Swagger specification. A lack of detail or ambiguity in these descriptors could potentially undermine the effectiveness of our approach. Additionally, the inclusion of a configurable mechanism to determine the volume of test data items, while resulting in a single prompt, could conceivably lead to incongruities when confronted with an overwhelming demand for test data. Such instances might sway the results towards simpler strategies to align with the desired outcomes.

ACKNOWLEDGEMENTS

This research is partially funded by Vietnam’s National Foundation for Science and Technology Development (NAFOSTED) under grant number NCUD.02-2019.72. This research is supported by research funding from Faculty of Information Technology, University of Science, Vietnam National University - Ho Chi Minh City.

REFERENCES

- [1] Juan C Alonso. 2021. Automated generation of realistic test inputs for web APIs. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1666–1668.
- [2] Fuad Sameh Alshraiedeh and Norliza Katuk. 2021. A URI parsing technique and algorithm for anti-pattern detection in RESTful Web services. *International Journal of Web Information Systems* 17, 1 (2021), 1–17.
- [3] Andrea Arcuri. 2019. RESTful API automated test case generation with EvoMaster. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 28, 1 (2019), 1–37.
- [4] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. 2019. Restler: Stateful rest api fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 748–758.
- [5] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K Lahiri. 2022. Toga: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering*. 2130–2141.
- [6] Hamza Ed-Douibi, Javier Luis Cánovas Izquierdo, and Jordi Cabot. 2018. Automatic generation of test cases for REST APIs: A specification-based approach. In *2018 IEEE 22nd international enterprise distributed object computing conference (EDOC)*. IEEE, 181–190.
- [7] Patrice Godefroid, Bo-Yuan Huang, and Marina Polishchuk. 2020. Intelligent REST API data fuzzing. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 725–736.
- [8] Zac Hatfield-Dodds and Dmitry Dygalo. 2022. Deriving semantics-aware fuzzers from web api schemas. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 345–346.
- [9] Stefan Karlsson, Adnan Čaušević, and Daniel Sundmark. 2020. QuickREST: Property-based test generation of OpenAPI-described RESTful APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 131–141.
- [10] Myeongsoo Kim, Davide Corradini, Saurabh Sinha, Alessandro Orso, Michele Pasqua, Rachel Tzoref-Brill, and Mariano Ceccato. 2023. Enhancing REST API Testing with NLP Techniques. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1232–1243.
- [11] Nuno Laranjeiro, João Agnelo, and Jorge Bernardino. 2021. A black box tool for robustness testing of REST services. *IEEE Access* 9 (2021), 24738–24754.
- [12] Yi Liu, Yuekang Li, Gelei Deng, Yang Liu, Ruiyuan Wan, Runchao Wu, Dandan Ji, Shiheng Xu, and Minli Bao. 2022. Morest: model-based RESTful API testing with execution feedback. In *Proceedings of the 44th International Conference on Software Engineering*. 1406–1417.
- [13] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651* (2023).
- [14] Alberto Martín-López, Sergio Segura, and Antonio Ruiz-Cortés. 2020. RESTest: Black-box constraint-based testing of RESTful web APIs. In *Service-Oriented Computing: 18th International Conference, ICSOC 2020, Dubai, United Arab Emirates, December 14–17, 2020, Proceedings* 18. Springer, 459–475.
- [15] OpenAI. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).
- [16] Emanuele Viglianisi, Michael Dallago, and Mariano Ceccato. 2020. Resttest-gen: automated black-box testing of restful apis. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 142–152.
- [17] Huayao Wu, Lixin Xu, Xintao Niu, and Changhai Nie. 2022. Combinatorial testing of restful apis. In *Proceedings of the 44th International Conference on Software Engineering*. 426–437.
- [18] Huayao Wu, Lixin Xu, Xintao Niu, and Changhai Nie. 2022. Combinatorial testing of restful apis. In *Proceedings of the 44th International Conference on Software Engineering*. 426–437.
- [19] Frank F Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 1–10.
- [20] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601* (2023).
- [21] Man Zhang and Andrea Arcuri. 2021. Adaptive hypermutation for search-based system test generation: A study on rest apis with evomaster. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 1 (2021), 1–52.